30/12/2025

Work done

- **Inheritance**

Description

Inheritance :

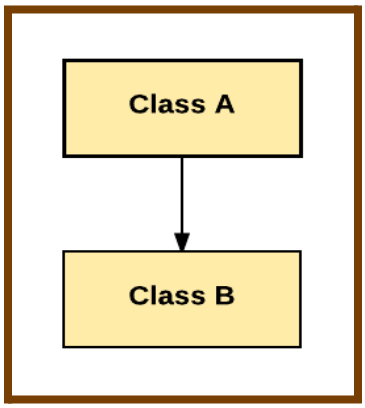
➔ Inheritance is an OOP concept where a child class acquires properties and methods of the parent class.

Advantages

➔ Code reusability
➔ Avoids duplication
➔ Easy maintenance
➔ Supports hierarchical design
➔ Represents real-world relationships

Types of inheritance :

1. **Single inheritance**
   ➔ Single child class inherits from single parent class.



There are two classes named class A and class B. Class B inherits from class A that means all the properties and behaviour of class A can use class B.

Syntax :
class A :
    Pass

class B(A) :
pass

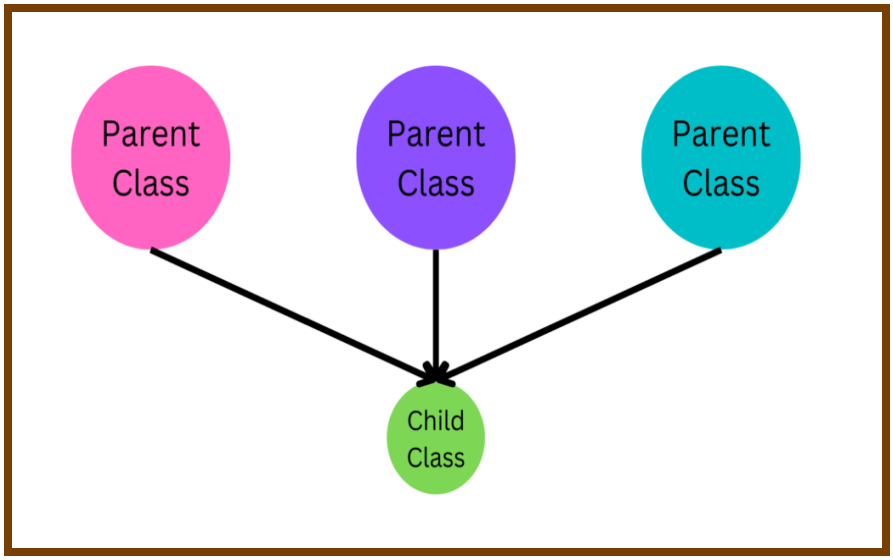
```
class A :
    def __init__(self,name):
        self.name = name
    def show(self):
        print("hello " + self.name + " !!")

class B(A) :
    pass

b1 = B("Sayma")
b1.show()
```

Output:
hello Sayma !!

```
class A :
    def __init__(self,name):
        self.name = name
    def show(self):
        print("hello " + self.name + " !!")

class B(A) :
    def display(self):
        print("it's child class..")

b1 = B("Sayma")
b1.show()
b1.display()
```

Output :
hello Sayma !!
it's child class..

2. **Multiple inheritance**
   - ➔ Single base class inherits from multiple parent class

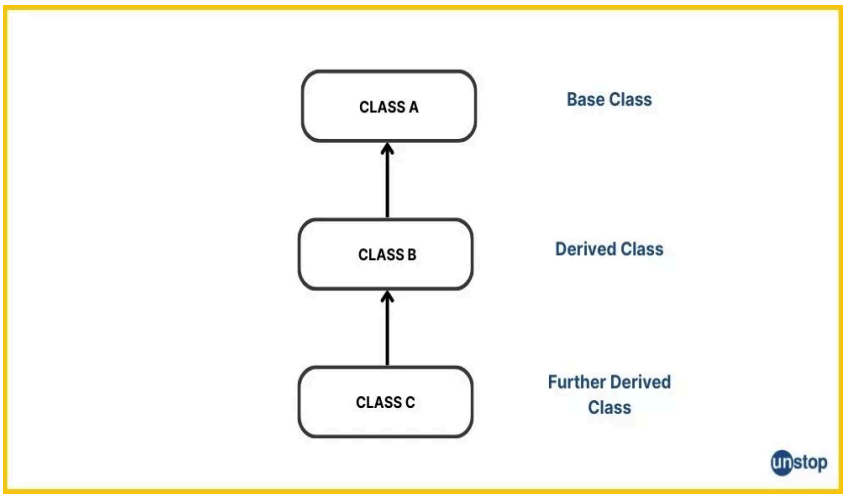



```
class A :
    def show1(self):
        print("It is first parent class ")

class B() :
    def show2(self):
        print("It is second parent class")

class C() :
    def show3(self) :
        print("It is third parent class")

class D(A,B,C):
    def show4(self):
        print("IT is child class")

d1 = D()
d1.show1()
d1.show2()
d1.show3()
d1.show4()
```

Output:
It is first parent class
It is second parent class
It is third parent class

IT is child class

3. **Multilevel inheritance**
   - ➔ Child class derived from another child class

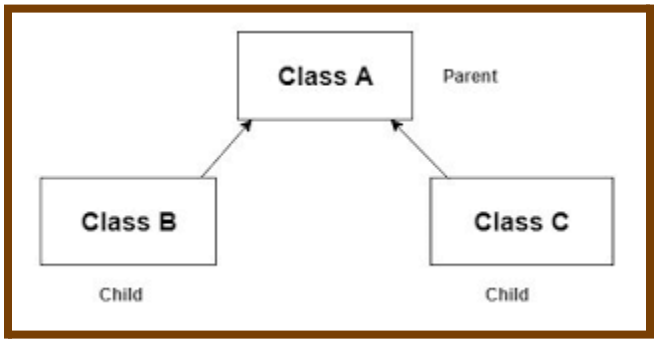



```
class A:
    def display(self):
        print("hello !!")

class B(A):
    def display1(self):
        print("I'm Sayma ")

class C(B) :
    def display2(self):
        print("I'm pursuing diploma in IT")

c1 = C()
c1.display()
c1.display1()
c1.display2()
```

Output:
hello !!
I'm Sayma
I'm pursuing diploma in IT

4. **Hierarchical inheritance**

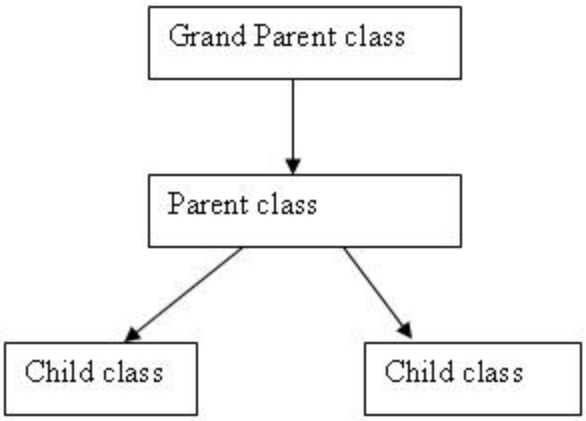
➔ Multiple derived class inherits from single base class




```
class person:
    def show1():
        print("this class is for person")

class student(person):
    def show2():
        print("this class for students")

class teacher(person):
    def show3():
        print("this class for teachers")

s1 = student()
s1.show1()
s1.show2()

t1 = teacher()
t1.show1()
t1.show3()
```

Output:
this class is for person
this class for students
this class is for person
this class for teachers

5. **Hybrid inheritance**
   ➔ Combination of two or more inheritance.




```
class A :
    def show1(self):
        print("class A")

class B(A) :
    def show2(self) :
        print("class B")

class C(A) :
    def show3(self) :
        print("class C")

class D(B,C):
    def show4(self):
        print("class D")

d1 = D()
d1.show1()
d1.show2()
d1.show3()
d1.show4()
```

Output :
class A
class B
class C
class D

Using __init__ in inheritance

```
class car :
    def __init__(self,name):
        self.name = name

class BMW(car):

    def show(self):
        print(self.name)

b1 = BMW("BMW")
b1.show()
```

Output:
BMW

super() method -

- ➔ The child class has __init__() function and the parent class also has __init__ function then python confused which constructor should they call.
- ➔ super() built-in function, For accessing parent instance attributes.

```
class car :
    def __init__(self,name):
        self.name = name

class BMW(car):
    def __init__(self, name , model):
        super().__init__(name)
        self.model = model

    def show(self):
        print(self.name,self.model)

b1 = BMW("BMW","M8")
b1.show()
```

Output:
BMW M8

```python
class animal:
    def sound(self):
        print("Animal sound")

class dog(animal):
    def dog_sound(self):
        super().sound()
        print("Dog barks")

d1 = dog()
d1.dog_sound()
```

Output:
Animal sound
Dog barks

Method overriding

```python
class employee:
    def work(self):
        print("employee works")

class intern(employee):
    def work(self):
        super().work()
        print("intern works")

i1 = intern()
i1.work()
```

Output:
employee works
intern works